PLATFORM FOR AUTONOMOUS INDOOR NAVIGATION USING SLAM, META-RL, AND MULTI-ROBOT COLLABORATION

25-26J-497

Project Proposal Report

Jayarathne H C D

BSc (Hons) Degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

Sri Lankan Institute of Information Technology Sri Lanka

August 2025

PLATFORM FOR AUTONOMOUS INDOOR NAVIGATION USING SLAM, META-RL, AND MULTI-ROBOT COLLABORATION

25-26J-497

Project Proposal Report

Jayarathne H C D– IT22311290

Supervisor: Mr. Samadhi Rathnayake

Co-Supervisor: Mr. Amila Alexander

BSc (Hons) Degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

August 2025

DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Jayarathne H C D	IT22311290	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

28/8/25

Signature of the Supervisor

Date

Signature of the Co-Supervisor

Date

ABSTRACT

The deployment of autonomous multi-robot systems in dynamic environments creates a significant challenge in operational oversight, debugging, and human-in-the-loop control. As these systems scale, the ability for human operators to monitor, understand, and trust their behavior becomes paramount. This research addresses this challenge by proposing a unified 3D visualization and debugging platform designed to serve as the central interface for a multi-robot navigation system that combines Simultaneous Localization and Mapping (SLAM) and Meta-Reinforcement Learning (Meta-RL).The platform is engineered to process and render data from a fleet of robots operating in a decentralized, cloud-assisted architecture. It ingests real-time telemetry and fused 3D maps generated from multi-robot SLAM data, providing a live, comprehensive operational picture. Unlike conventional passive viewers, this platform integrates interactive debugging and analytics tools, allowing operators to move beyond simple monitoring. Key features include the visualization of historical collision heatmaps, SLAM accuracy metrics, and the ability for users to directly interact with the map by adding annotations or defining operational zones.A dedicated web-based interface provides operators with real-time access to robot locations, sensor data, and the unified environmental map. This intuitive interface is designed to support not only live monitoring but also to create a shared operational picture for both technical teams and end-users. This ensures that domain expertise can be leveraged alongside automated learning, leading to more robust and context-aware system management.The primary contributions of this platform are enhanced situational awareness, streamlined system diagnostics, and a significant reduction in the complexity of managing autonomous fleets. By bridging the gap between complex robotic perception and human operational oversight, this platform makes the deployment of multi-robot systems more practical, trustworthy, and manageable for industrial, public safety, and research applications.

Keywords: Multi-Robot Systems, 3D Visualization, SLAM, Operational Oversight, Interactive Debugging

TABLE OF CONTENT

Contents

Declaration i

Abstract ii

List of tables v

List of figures vi

List of abbreviations vii

1 Introduction 1

1.1 Background and Literature Survey 3

1.2 Research Gap 6

1.3 Research Problem 9

2 Research Objectives 11

2.1 Main Objectives 11

2.2 Specific Objectives 12

2.2.1 Design and Implement a Modular SLAM Backend 12

2.2.2 Develop an Interactive 3D Visualization Client 12

2.2.3 Engineer a Low-Latency Data Synchronization Gateway 12

2.2.4 Integrate Advanced Debugging and Analytics Tools 12

2.2.5 Build a User-Interactive Annotation System 13

..... 13

3 Methodology 14

3.1 System Overview 14

3.2 Individual Component 15

3.3 Software Solutions 16

3.3.1 Requirement Gathering and Analysis 16

3.3.2 Feasibility Study 17
3.4 Tasks and Sub-Tasks 18
3.5 Testing and Validation 19
3.6 Data Collection 20
3.7 Time Frame & Schedule 21
4 Project requirements 22
4.1 Functional Requirements 22
4.2 Non-Functional Requirements 22
4.3 User Requirements 23
4.4 System Requirements 23
5 REFERENCES 24

LIST OF TABLES

Table 1.1 Core Challenges in Multi-Robot System Deployment..... 5
Table 2 Comparative Analysis of Existing Visualization Approaches..... 7

LIST OF FIGURES

Figure 3.1 System Overview Diagram 14
Figure 3.2 Component Architecture of the Visualization Platform 15
Figure 3.3 Gantt Chart 21

LIST OF ABBREVIATIONS

Abbreviation	Description
AGV	Automated Guided Vehicle
AMR	Autonomous Mobile Robot
API	Application Programming Interface
CPU	Central Processing Unit
DRL	Deep Reinforcement Learning
FPS	Frames Per Second
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
Meta-RL	Meta-Reinforcement Learning
RAM	Random-Access Memory
SLAM	Simultaneous Localization and Mapping
UAT	User Acceptance Testing
WebGL	Web Graphics Library

1 INTRODUCTION

The growing demand for autonomous navigation in dynamic indoor environments, such as automated warehouses and disaster recovery zones, presents significant robotics challenges. While Simultaneous Localization and Mapping (SLAM) has enabled robots to operate without prior maps by reconstructing environments in real-time [1], traditional SLAM algorithms often assume static surroundings. This limitation hinders their effectiveness in real-world applications where dynamic elements like moving people, temporary obstacles, and reflective surfaces can lead to map inconsistencies and localization failures, compromising both safety and efficiency [4].

To operate effectively in these unpredictable settings, robots require two further capabilities: rapid adaptation and scalable collaboration. Standard rule-based path planning often fails in dynamic conditions, creating a need for more intelligent strategies [5]. Meta-Reinforcement Learning (Meta-RL) has emerged as a powerful solution, enabling robots to leverage past experiences to quickly adapt their navigation policies to new and unseen situations [9]. However, scaling from a single adaptive robot to a multi-robot team introduces complexities in coordination. Collaborative navigation offers immense benefits, including efficient mapping through shared exploration and greater resilience [6]. Achieving this requires a robust framework for real-time map sharing, task allocation, and conflict resolution among heterogeneous robot teams [7], [8].

This proposal introduces an integrated platform that holistically addresses these challenges by combining advanced SLAM, a Meta-RL based adaptation engine, and a decentralized multi-robot collaboration framework. The system utilizes a modern edge-cloud architecture, where lightweight robots perform basic sensing while offloading computationally intensive tasks—like SLAM processing and policy optimization—to a centralized server. This design ensures scalability, cost-efficiency, and interoperability across diverse robotic platforms [6].

However, a purely autonomous system is incomplete without effective human oversight. A critical gap in deploying such complex systems is the ability for operators

to monitor, debug, and ultimately trust their behavior. This research directly addresses this gap through the development of the 3D Mapping & Visualization Platform, which serves as the essential, user-friendly interface for real-time monitoring and diagnostics of the entire multi-robot operation. By integrating this vital visualization component with the core autonomous systems, our project enables multi-robot teams to navigate collaboratively, adapt to dynamic changes, and maintain consistent global maps in complex indoor environments. The suggested solution bridges the gap between academic research and practical implementation, increasing logistical efficiency and enabling new robotics applications in human-centric environments.

.

1.1 Background and Literature Survey

The evolution of autonomous systems has altered the way indoor logistics, inspection and service activities are carried out. Historically, tasks like item transportation and inventory management relied significantly on human labor which was inefficient, physically demanding and error prone. Automated Guided Vehicles (AGVs) were the first wave of automation to enable repeated activities in organized and controlled environments. However, AGVs depended on permanent navigation infrastructure, such as magnetic strips or painted floor lines, rendering them inefficient in the places with layout changes and dynamic obstacles [1].

The introduction of Autonomous Mobile Robots (AMRs) marked a significant leap forward. AMRs can dynamically detect and traverse new surroundings by utilizing advanced sensing methods such as LiDAR, RGB-D cameras and inertial measurement units (IMUs) as well as SLAM algorithms [3]. This enables its usage in complicated, real-world environments such as warehouses, hospitals and smart buildings. However, in large-scale applications a single AMR is insufficient. The true potential lies in deploying fleets of robots that can work together to explore and operate more effectively. This transition from a single agent to multi agent system creates a new set of challenges that define the boundary of current robotics research [7].

Indoor environments are hardly static because moving people, rearranging furniture and temporary obstacles cause a shift in distribution between training and operating settings. While Deep Reinforcement Learning (DRL) has shown ordinary DRL agents require considerable training in new settings, making them not suitable for applications that require immediate attention. Meta-RL solves this problem by allowing agents to "learn how to learn" and change rules with little new knowledge [10]. For example, Google research conducted in 2020 demonstrated fast adaptability in legged robots through evolutionary meta-learning [11]. However, developing whole lifecycle platforms from simulation-based training to real world deployments and constant fine-tuning is still an unexplored topic.

Scaling from a single robot to a collaborative team raises serious distributed system challenges. To outperform a single agent robot, a team of robots must coordinate their

activities to achieve a shared goal, such as exploring an unexplored region. This requires addressing several key challenges:

- **Communication and Data Fusion**: Agents must effectively share sensor data and local maps with each other. Combining these incomplete maps into a single global map is a difficult data fusion task [6]. Systems such as CORB-SLAM have created frameworks for this but they frequently rely on protocols that may struggle with scalability and network instability.
- **Dynamic Task Allocation**: Dynamic task allocation is appointing different agents for different areas of a building to explore in order to increase coverage and reduce redundancy. Cooperative navigation techniques are vital for efficiency but building decentralized algorithms that lead to globally optimum behavior remains a persistent difficulty [8].
- **Conflict Resolution**: Managing path conflicts and right-of-way to prevent collisions and deadlocks in shared environments.

These challenges require robust communication protocols, consensus algorithms, and lightweight yet scalable data-sharing strategies.

A multi-robot fleet generates a continuous stream of high-dimensional real-time data which includes raw sensor feeds, SLAM-generated maps, navigation choices, job allocations and inter-robot communication. Without the appropriate tools, this data can become overwhelming for operators, which can make the system impossible to analyze, diagnose and optimize. Existing interfaces often provide only basic 2D positional data, lacking advanced features such as collision heatmaps, historical performance analytics, and interactive map annotations [12].

The absence of integrated visualization and diagnostic tools in many existing multi-robot frameworks is a major obstacle to scaling. Without them, troubleshooting involves manual log analysis, which is time-consuming and prone to error.

To address this difficulty, a 3D Mapping & Visualization Platform must be developed that not only depicts real-time global maps and robot data, but also includes historical analytics, performance heatmaps, and interactive controls. Such a system converts raw sensor and navigation data into usable insights, allowing operators to make

quick decisions, save downtime, and retain confidence in the system's performance even if the number of robots and mission complexity increases.

Table 1.1 Core Challenges in Multi-Robot System Deployment

Factor	Practical Effect on Robots	Typical Symptom	Impact (1-5)
Real-Time Adaptation and Dynamic Obstacles	Inability to adjust to new or moving obstacles	Collisions, inefficient pathing, operational freezes	5
Multi-Robot Coordination and Scalability	Robots performing redundant work or interfering with each other	Inefficient area coverage, inconsistent global maps, bottlenecks	5
System Complexity and Operational Oversight	Inability for human operators to monitor, debug, or manage the fleet	Prolonged downtime, difficulty diagnosing failures, lack of trust in the system	4

The available literature typically addresses these issues in isolation. Meta-RL research may produce a highly adaptable agent, but it fails to investigate how it operates in multi-agent systems. Working on collaborative SLAM may help with map merging, but it assumes robots with basic, non-adaptive behaviors. The key gap therefore is the lack of a single, integrated platform to address these concerns comprehensively. Without a strong framework to regulate these interactions, a team of robots may be less efficient than a single agent. This difficulty is essentially a distributed systems engineering problem, requiring solutions that assure data consistency and allow for dynamic resource allocation across independent agents.

1.2 Research Gap

Autonomous navigation is a rapidly evolving field, with significant progress in core technologies like SLAM, reinforcement learning, and multi-robot collaboration. While individual studies on systems like ORB-SLAM, RTAB-Map, and Meta-RL frameworks like MAML demonstrate powerful capabilities, this progress has inadvertently created a new and critical research gap: the operational oversight gap. As robotic systems become more autonomous, distributed, and adaptive, their internal states and decision-making processes become increasingly opaque. This oversight gap is a major barrier to practical deployment, as it leads to inefficient troubleshooting, a lack of trust in autonomous decision-making, and an inability to guarantee operational safety in complex, real-world scenarios.

The specific limitations of these foundational technologies directly fuel the need for advanced oversight. For instance, even state-of-the-art SLAM systems struggle in highly dynamic environments where moving obstacles can corrupt the map, creating a reality that no longer matches the robot's internal model. Without a way for an operator to visualize these mapping inconsistencies in real-time, the entire fleet may operate on faulty data. Similarly, while Meta-RL techniques promise rapid adaptation, their learned policies can be unpredictable, creating a critical gap between simulation-trained behaviors and real-world performance. An operator requires robust diagnostic tools to understand *why* a robot is behaving unexpectedly, which is essential for debugging and building trust in the system.

Furthermore, scaling to collaborative multi-robot teams using frameworks like CORB-SLAM and DDF-SLAM generates an exponential increase in data complexity. Operators must track the positions, statuses, and sensor feeds from multiple agents simultaneously, while also trying to comprehend a global map that is being constantly fused from different, sometimes conflicting, sources. This data deluge makes manual monitoring impossible and highlights the need for a platform that can intelligently synthesize this information into a single, comprehensible operational picture.

Current visualization and management tools are ill-equipped for this integrated challenge. Developer-centric tools, while powerful for single-robot debugging, are not designed for fleet-level operational oversight by non-expert users and lack historical analysis capabilities. Conversely, commercial fleet management systems focus on high-level logistics, often providing only basic 2D positional data and lacking the deep diagnostic capabilities needed to analyze fused 3D maps or environmental changes [12]. The most significant deficiency is the absence of a platform that connects deep robotic perception with high-level operational management. While research exists for dynamic change detection algorithmically [8], [9], these methods are typically offline or single-robot focused and are not integrated with a real-time visualization front-end. No existing tool provides a single interface that integrates:

- Real-time, fused 3D maps from a heterogeneous multi-robot team.
- Advanced diagnostics, such as historical collision heatmaps and SLAM accuracy metrics.
- Interactive, version-controlled visualization of environmental changes.
- Human-in-the-loop feedback mechanisms, such as semantic map annotation.

Table 2 Comparative Analysis of Existing Visualization Approaches

Approach	Strengths	Limitations / Gap	How Proposed Platform Fills the Gap
Standard SLAM Viewers	Good for real-time, single-robot developer debugging.	Not designed for fleet management; no historical analytics; complex for non-experts.	Provides a user-friendly interface for multi-robot fleet monitoring with integrated

			historical analytics (e.g., heatmaps).
Commercial Fleet Managers	Strong focus on high-level logistics (tasking, battery).	Typically limited to 2D maps; lack deep SLAM diagnostics or environmental change visualization [12].	Offers rich 3D visualization of fused maps and integrates advanced SLAM performance metrics.
Change Detection Research	Effective at identifying environmental changes algorithmically [8], [9].	Algorithms are often offline and lack an integrated, real-time visualization interface for operators.	Implements a real-time "map diff" tool and version history, allowing operators to see environmental changes as they happen.

Therefore, the central gap this research addresses is not a deficit in any single autonomous algorithm, but in the critical interface that connects the complex multi-robot system to its human supervisors. The proposed system addresses this integration gap by providing a holistic platform that unifies these capabilities for the first time. By developing a comprehensive 3D Mapping & Visualization Platform that integrates real-time monitoring, advanced diagnostics, and interactive annotation, this project will provide the necessary tools to make the management of autonomous fleets more transparent, trustworthy, and efficient, overcoming a key limitation of existing research.

1.3 Research Problem

The rapid advancement of autonomous mobile robots (AMRs) has significantly improved indoor navigation capabilities, yet numerous challenges remain that hinder their deployment in dynamic and unstructured environments. Traditional Simultaneous Localization and Mapping (SLAM) techniques, such as ORB-SLAM [1] and RTAB-Map [2], have demonstrated high-accuracy mapping and localization in static or structured settings; however, their performance significantly declines when faced with dynamic obstacles, moving furniture, or temporary environmental changes.

Similarly, LiDAR-based mapping systems like Cartographer [3] provide real-time 2D and 3D mapping but are computationally intensive and lack the ability to adapt effectively to real-time dynamic conditions or integrate semantic awareness. While these SLAM systems form the backbone of robotic perception, they typically operate in isolation and do not natively support decentralized multi-robot collaboration or adaptive navigation strategies. In parallel, reinforcement learning approaches, particularly Meta-Reinforcement Learning (Meta-RL) techniques like MAML [4], have enabled rapid adaptation to unseen scenarios, allowing robots to generalize navigation policies to new environments.

Studies such as Zeng et al. [5] demonstrate the potential of Meta-RL for obstacle avoidance in simulated environments. However, these methods are primarily simulation-based, often ignoring real-world constraints like sensor noise, communication delays, and the latency inherent in real-time control. Furthermore, existing Meta-RL implementations rarely consider multi-robot environments where coordination, task allocation, and map-sharing are critical. Multi-robot collaboration frameworks, including CORB-SLAM [6] and DDF-SLAM [7], offer distributed SLAM solutions with map fusion and shared environmental awareness, enhancing global consistency and exploration speed. Nonetheless, these systems rely on centralized task allocation, assume homogeneous robot teams, and lack integration with adaptive Meta-RL policies for real-time obstacle-aware navigation, limiting their scalability and robustness in heterogeneous or large-scale deployments. Another persistent challenge is

dynamic change detection. Approaches by Sünderhauf et al. [8] and Kim et al. [9] incorporate multi-sensor fusion and semantic information to detect environmental changes over time, improving map accuracy and navigation safety. Yet, most existing methods are offline or designed for single-robot scenarios, failing to support real-time, decentralized semantic-aware updates in multi-robot teams. As a result, robots often operate with outdated maps, leading to inefficiencies and potential collisions. Collectively, these limitations highlight the lack of a unified solution that seamlessly integrates SLAM, Meta-RL adaptive navigation, multi-robot collaboration, and dynamic change detection into a comprehensive autonomous navigation platform. Such a platform is essential for enabling AMRs to navigate unknown indoor environments efficiently, adapt to dynamic obstacles in real time, share mapping and exploration data among heterogeneous teams, and maintain continuously updated semantic-aware maps.

Addressing this problem requires a holistic approach that combines scalable software architectures, hybrid edge-cloud processing, robust communication protocols, and modular algorithmic pipelines. By integrating these capabilities, the proposed platform aims to provide an end-to-end solution that ensures real-time adaptability, operational robustness, and scalability, bridging the gap between isolated research contributions and practical deployment in real-world dynamic environments. This research problem directly motivates the design and implementation of a comprehensive autonomous navigation system capable of synergizing 3D SLAM mapping, Meta-RL-based obstacle-aware adaptation, decentralized multi-robot coordination, and semantic-aware change detection, thereby addressing the critical limitations present in the current state of the art.

2 RESEARCH OBJECTIVES

2.1 Main Objectives

The primary aim of this research is to design, develop, and validate a comprehensive autonomous navigation platform capable of operating efficiently in dynamic and unknown indoor environments. The platform will seamlessly integrate adaptive navigation algorithms, robust mapping systems, and multi-robot collaboration strategies into a unified ecosystem. This will involve the combination of advanced SLAM-based mapping, adaptive obstacle avoidance using reinforcement learning techniques, and a decentralized coordination mechanism to ensure scalability and robustness in multi-agent deployments. Furthermore, the platform will incorporate semantic change detection to maintain an up-to-date understanding of the environment, enabling the robots to make informed navigation and task allocation decisions in real time. The system will not only exist in simulation but will also be implemented and tested on a functional prototype robot, bridging the gap between theory and practical application. In addition to core navigation functionalities, the research aims to establish a strong model training pipeline and deployment strategy that allows rapid iteration and improvement of algorithms while ensuring they meet real-world operational constraints.

Real-time monitoring tools will be integrated into the system to continuously assess model performance, detect anomalies, and trigger adaptive retraining when necessary. By unifying training, deployment, monitoring, and physical testing into a single framework, the research intends to provide a scalable, reliable, and adaptable solution that can serve as a benchmark for future developments in autonomous navigation for both industrial and service robotics contexts.

2.2 Specific Objectives

2.2.1 Design and Implement a Modular SLAM Backend

The first objective is to design and implement a scalable, server-side backend capable of processing and managing data from a fleet of robots. This backend will feature a modular, pluggable architecture with well-defined APIs to ingest sensor data from multiple sources. It will be responsible for fusing partial maps from individual robots into a consistent global 3D map, storing historical data for later analysis, and implementing the core change detection algorithms to compare map versions and identify environmental modifications.

2.2.2 Develop an Interactive 3D Visualization Client

The second objective is to create a high-performance, web-based interface for visualizing the multi-robot system in real time. Using modern 3D rendering libraries, this client will display the fused SLAM-generated maps and live telemetry from all robots. The focus will be on creating an intuitive and responsive user experience that allows operators to easily navigate complex 3D scenes and understand the state of the system at a glance.

2.2.3 Engineer a Low-Latency Data Synchronization Gateway

The third objective is to engineer an efficient, low-latency communication gateway to stream data from the backend to the visualization client. This real-time link is critical for providing operators with up-to-the-second information on robot positions, alerts about detected map changes, and other system notifications. This will involve implementing modern communication protocols like WebSockets and optimizing data serialization to minimize bandwidth and ensure a smooth user experience.

2.2.4 Integrate Advanced Debugging and Analytics Tools

The fourth objective is to move beyond simple monitoring by integrating advanced diagnostic and analytical tools into the platform. This includes developing features for visualizing historical data, such as collision heatmaps,

and providing a version-controlled "map diff" tool to visually compare different states of the environment. This will allow operators to see exactly what has changed over time, such as new obstacles or altered pathways, which is critical for debugging and maintaining trust in the system's perception.

2.2.5 Build a User-Interactive Annotation System

The final objective is to empower operators with the ability to interact directly with the environmental map. This involves building tools within the interface that allow users to tag known obstacles, define operational zones (e.g., "keep-out" areas), or manually confirm or reject changes detected by the backend. This feature creates a human-in-the-loop feedback mechanism, enhancing the semantic understanding of the environment for both the robots and human operators.

3 METHODOLOGY

3.1 System Overview

The proposed system is an integrated platform for Autonomous Indoor Navigation that utilizes an edge-cloud architecture. Lightweight robots equipped with basic sensors operate at the edge, performing immediate local tasks like obstacle avoidance while streaming sensor data to a centralized server. This design allows for heavy computations such as SLAM processing, multi-robot map fusion, and Meta-RL policy optimization to be offloaded to the cloud. This approach ensures the system is scalable, cost-efficient, and interoperable with heterogeneous robot teams while maintaining real-time performance. The overall architecture, as shown in the system diagram, consists of three main interacting components: the Meta-RL Adaptation Engine, the Multi-Robot Collaboration Framework, and the 3D Mapping & Visualization Platform.

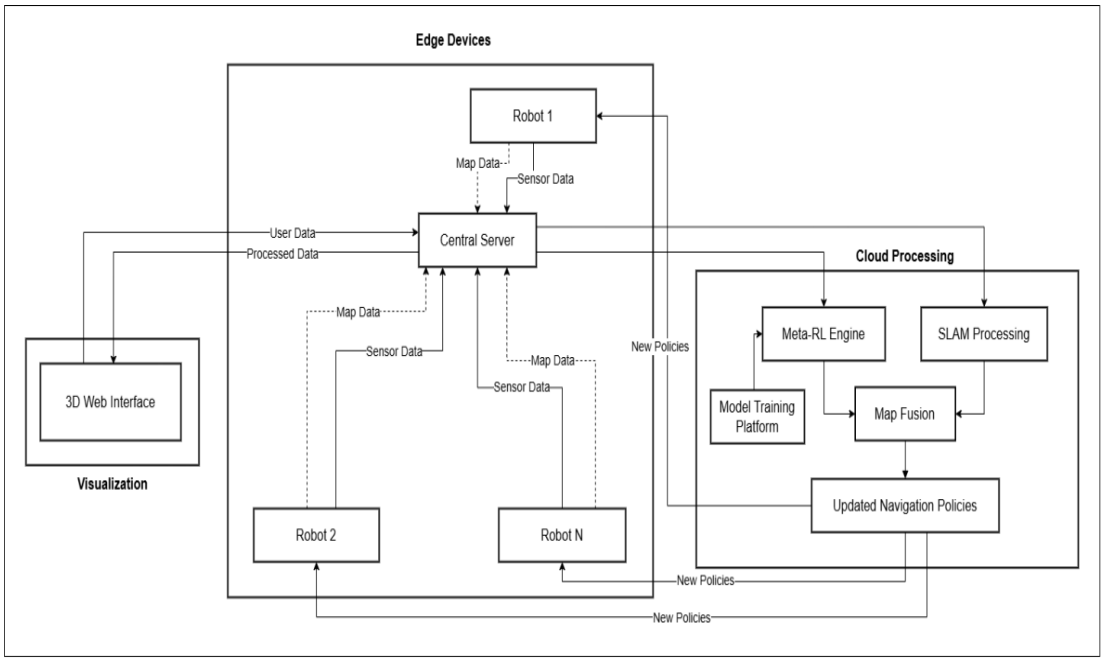

Figure 3.1 System Overview Diagram

3.2 Individual Component

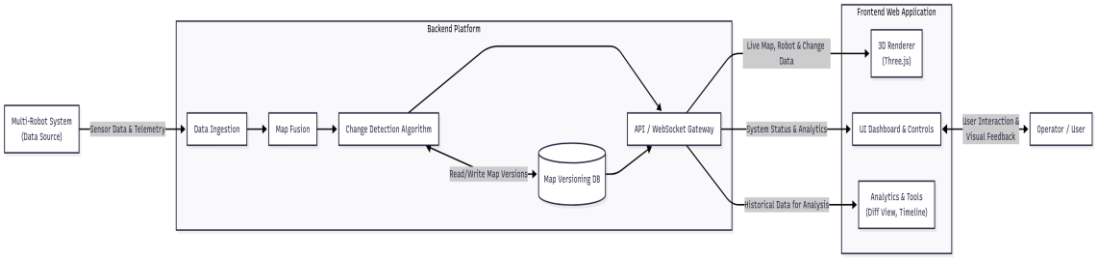

Figure 3.2 Component Architecture of the Visualization Platform

This component, the 3D Mapping & Visualization Platform, serves as the central hub for environmental state management and human operational oversight. It is a full-stack application with two primary parts: a backend for data processing and a frontend for visualization.

The backend is responsible for receiving sensor data from the entire robot fleet via the central server. Its first task is to fuse the partial maps from each robot into a single, consistent global map. The core logic then resides in its change detection pipeline. This pipeline continuously compares new incoming data against the current version of the global map to identify structural changes. If a change is validated, the backend creates and stores a new, time-stamped version of the map, effectively maintaining a historical record of the environment.

The frontend is an interactive, web-based 3D client. It communicates with the backend via a low-latency gateway to receive and render the live global map and the real-time positions of all robots. Its most critical feature is the visualization of the data produced by the change detection backend. Operators can use tools like a "map diff" view to see highlighted changes between versions or use a timeline to review the environment's evolution. This tight integration of backend detection and frontend visualization makes the system a powerful tool for monitoring and diagnostics.

3.3 Software Solutions

3.3.1 Requirement Gathering and Analysis

The first step in the 3D Mapping & Visualization Platform's requirement gathering process is to define the technical specifications for data rendering, user interaction, and integration with the central server. Functional requirements, such as real-time 3D map rendering, live multi-robot telemetry display, and interactive debugging tools (e.g., collision heatmaps, map version comparison), are prioritized. These are assessed alongside non-functional requirements like high frame rates (FPS), low data latency, scalability to handle data from a growing fleet, and operator usability.

Primary data sources for this process include a comprehensive review of academic literature on human-robot interaction, data visualization, and operational dashboards. Industrial case studies from drone fleet management and industrial control rooms provide valuable insights into designing effective user interfaces for complex systems. These requirements are further refined through consultations with robotics engineers and potential end-users to ensure the platform meets practical operational needs. Early-stage prototyping with mock data and wireframes helps establish initial performance baselines for rendering large 3D maps and measuring the responsiveness of the user interface.

This process also involves a detailed analysis of operational limitations to ensure the platform's robustness and adaptability:

- **High Volume of Incoming Data:** The platform's backend must be designed to ingest and process continuous, high-bandwidth streams of fused map data and telemetry from the central server. This necessitates an efficient data processing pipeline and scalable architecture to prevent bottlenecks and ensure real-time updates.
- **Variable User Network Conditions:** The web-based client will be accessed over networks with varying quality. The platform must implement adaptive data streaming and compression techniques to provide a smooth, responsive user experience even on less reliable connections.

- **Graceful Degradation and Server Failure:** In the event of a backend or central server failure, the frontend client must not crash. It should enter a "disconnected" state, clearly inform the user of the issue, and attempt to re-establish connection automatically. Any user-generated data, such as annotations, should be cached locally to prevent loss.

By addressing these factors, the requirement gathering phase establishes a solid foundation for a 3D Mapping & Visualization Platform that is both powerful and reliable in real-world operational scenarios.

3.3.2 Feasibility Study

A feasibility study is essential to determine if the proposed architecture for the 3D Mapping & Visualization Platform can meet performance, budget, and deployment constraints. This evaluation provides a comprehensive assessment from multiple perspectives.

- **Technical Feasibility:** Confirming the compatibility of frontend web technologies (e.g., WebGL, Three.js) with target browsers for rendering complex 3D scenes. Evaluating backend frameworks (e.g., Python/Flask, Node.js/Express) for their ability to process large datasets and stream data efficiently via WebSockets.

- **Economic Feasibility:** Estimating costs for cloud infrastructure (server hosting, database storage), web hosting, and potential software licensing for specialized 3D libraries or UI components.

- **Operational Feasibility:** Assessing the complexity of integrating with the central server's data API. Evaluating the learning curve and training requirements for operators who will use the platform for monitoring and debugging tasks.

- **Schedule Feasibility:** Determining whether key development milestones—such as backend API completion, frontend client development, and full system integration—can be achieved within the project’s timeline.

3.4 Tasks and Sub-Tasks

To achieve the goals of the 3D Mapping & Visualization Platform, the development is broken down into major work packages, each with distinct tasks to ensure systematic progress.

1. Backend Architecture & Setup

- **Finalize Technology Stack:** Choose backend languages, frameworks, and database solutions based on performance and scalability requirements.
- **Setup Cloud Infrastructure:** Provision and configure servers, databases, and necessary network services.
- **Design Data Ingestion API:** Develop robust API endpoints for receiving live map and telemetry data from the central server.

2. Core Backend Module Development

- **Implement Map Versioning & Change Detection Engine:** Develop the core logic for comparing map versions and identifying environmental changes.
- **Develop Real-time Data Processing Pipeline:** Create the service that processes and prepares incoming data for streaming to the frontend.
- **Integrate Historical Analytics Module:** Build the functionality to process historical data for features like collision heatmaps.

3. Frontend Development & Integration

- **Develop Interactive 3D Rendering Client:** Build the main web interface using a 3D library like Three.js.
- **Implement Real-time Data Synchronization:** Engineer the WebSocket gateway to connect the frontend to the backend data stream.
- **Build UI for Interactive Tools:** Create the user controls for features like the map comparison slider and the annotation system.

4. Testing & Validation

- **Unit and Integration Tests:** Test backend modules and frontend components individually and together.
- **Performance and Load Testing:** Evaluate the system's responsiveness and stability under simulated high data loads.
- **User Acceptance Testing (UAT):** Conduct trials with end-users to gather feedback.

5. Documentation & Handover

- **System Architecture Documentation:** Create detailed documents for the backend and frontend architecture.
- **User Manual Creation:** Develop a guide for operators on how to use the platform.

3.5 Testing and Validation

Testing will be performed in progressive stages to ensure a robust and reliable platform:

- **Unit Testing:** Each software module (e.g., backend change detection, API endpoints, frontend rendering component) will be tested in isolation to verify its core functionality.
- **Integration Testing:** The end-to-end data flow from the central server, through the backend processing pipeline, to the frontend visualization client will be validated to ensure seamless interaction.
- **Performance Testing:** Key metrics such as API response time, data streaming latency, and frontend frames-per-second (FPS) while rendering complex scenes will be measured to assess efficiency and identify bottlenecks.
- **Usability Testing (UAT):** Target users will be asked to perform specific tasks on the platform, such as identifying a recent environmental change or

annotating a hazard. Feedback on the interface's clarity and ease of use will be collected.

- **Cross-Browser Compatibility Testing:** The web application will be tested on major browsers (e.g., Chrome, Firefox, Safari) to ensure consistent performance and appearance.

3.6 Data Collection

Data will be gathered to support the development and validation of the 3D Mapping & Visualization Platform.

- **Live Operational Data:** The primary data source will be the real-time stream of fused 3D maps, LiDAR point clouds, and robot telemetry (position, orientation) provided by the project's central server. This data is what the platform will ingest and visualize during operation.

- **Validation & Test Datasets:** To develop and validate the backend's change detection algorithm, datasets of indoor environments with known, timestamped changes will be required. This data can be sourced from publicly available robotics datasets or generated using a simulation environment to provide a "ground truth" for measuring the algorithm's accuracy.

3.7 Time Frame & Schedule

Figure 3.3 Gantt Chart

4 PROJECT REQUIREMENTS

4.1 Functional Requirements

- **Backend:**

  - Must ingest sensor data from multiple robots.
  - Must implement algorithms to compare new data against the current map to detect structural changes.
  - Must create, store, and manage versioned instances of the global map.
  - Must provide an API for the frontend to query map data, robot states, and change logs.

- **Frontend:**

  - Must render large-scale 3D maps and live robot positions.
  - Must visually highlight detected differences between map versions.
  - Must provide interactive tools (e.g., a timeline slider) to compare map history.
  - Must display alerts upon the detection of a critical environmental change.
  - Must allow users to add, edit, and view semantic annotations on the 3D map.

4.2 Non-Functional Requirements

- **Performance:** The backend change detection algorithm must process data in near real-time. The frontend client must maintain a high, smooth frame rate.
- **Accuracy:** The detection algorithm must achieve a high true-positive rate while minimizing false positives to ensure operator trust.
- **Scalability:** The architecture must scale to handle an increasing number of robots without performance degradation.

- **Usability:** The web interface must be intuitive, allowing operators to quickly understand the system's status and interpret detected changes with minimal training.
- **Maintainability:** The platform will be built with modular code and clear documentation to support future development and maintenance.

4.3 User Requirements

- Users must be able to view the real-time status and location of all robots on a single, unified 3D map.
- Users must be able to easily identify when, where, and how the environment has changed since the last observation.
- Users should be able to review the history of the environment's changes to understand trends or investigate incidents.
- Users must be able to add contextual information to the map through annotations.

4.4 System Requirements

- A modern web browser with WebGL support for the frontend client (e.g., Chrome, Firefox).
- A server (local or cloud-based) to host the backend application and database.
- Node.js and/or Python runtime environments for the backend services.
- A PostgreSQL or similar database for storing map versions and annotations.

5 REFERENCES

[1] Yu, C., Liu, Z., Liu, X.-J., Xie, F., Yang, Y., Wei, Q., and Fei, Q., "A Semantic Visual SLAM towards Dynamic Environments," *arXiv preprint arXiv:1809.08379*.

[2] Oriolo, G., Ulivi, G., and Vendittelli, M., "Real-time map building and navigation for autonomous robots in unknown environments," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 28, no. 3, pp. 316-333, 1998.

[3] Xing, Z., Zhu, X., and Dong, D., "DE-SLAM: SLAM for highly dynamic environment," *Journal of Field Robotics*, vol. 39, no. 4, pp. 528-549, 2022.

[4] Ai, Y., Rui, T., Yang, X., He, J., Fu, L., Li, J., and Lu, M., "Visual SLAM in dynamic environments based on object detection," *Journal of Manufacturing Systems*, vol. 60, pp. 257-271, 2021.

[5] Gan F, Xu S, Jiang L, Liu Y, Liu Q, Lan S. Robust visual SLAM algorithm based on target detection and clustering in dynamic scenarios. Front Neurorobot. 2024 Jul 23;18:1431897. doi: 10.3389/fnbot.2024.1431897. PMID: 39108349; PMCID: PMC11300432.

[6] Li, F., Yang, S., Yi, X., and Yang, X., "CORB-SLAM: A Collaborative Visual SLAM System for Multiple Robots," in *Proc. International Conference on Intelligent Robotics and Applications*, 2018, pp. 377-388.

[7] Ahmed, M. F. et al., "Efficient Frontier Management for Collaborative Active SLAM," *arXiv preprint arXiv:2310.01967*, 2023.

[8] Ducatelle, Frederick & Förster, Alexander & Bonani, Michael & Dorigo, Marco & Magnenat, Stéphane & Mondada, Francesco & O'Grady, Rehan & Pinciroli, Carlo & Retornaz, Philippe & Trianni, Vito & Gambardella, Luca Maria. (2014). Cooperative navigation in robotic swarms. Swarm Intelligence. 10.1007/s11721-013-0089-4.

[9] Wu X, Wang G and Shen N (2023) Research on obstacle avoidance optimization and path planning of autonomous vehicles based on attention mechanism combined with multimodal information decision-making thoughts of robots. *Front. Neurorobot.* 17:1269447. doi: 10.3389/fnbot.2023.1269447

[10] Raj, R. and Kos, A., "Intelligent mobile robot navigation in unknown and complex environment using reinforcement learning technique," *Scientific Reports*, vol. 14, no. 1, p. 22852, 2024.

[11] Fu, Z., Kumar, A., Malik, J., and Pathak, D., "Rapidly Adaptable Legged Robots via Evolutionary Meta-Learning," *Google Research*, 2020.

[12] Blanco, Jose Luis. (2019). A Modular Optimization Framework for Localization and Mapping. 10.15607/RSS.2019.XV.043.

[13] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A versatile and accurate monocular SLAM system,” *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.

[14] M. Labbé and F. Michaud, "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019. doi: 10.1002/rob.21831.

[15] W. Hess, D. Kohler, H. Rapp, and D. Andor, “Real-time loop closure in 2D LIDAR SLAM,” *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 1271–1278, 2016.

[16] C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 1126–1135, 2017.

[17] A. Zeng, S. Song, K. Yu, M. K. Wang, and T. Funkhouser, “Learning Synergistic Navigation Policies via Meta-Reinforcement Learning,” *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2563–2570, Apr. 2020.

[18] T. Huang, M. Kaess, and J. Leonard, “CORB-SLAM: Collaborative Range-Only SLAM for Multi-Robot Systems,” *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 281–288, 2019.

[19] M. Cieslewski, K. S. Liu, and D. Scaramuzza, “Data-Driven Decentralized SLAM for Heterogeneous Robot Teams,” *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2565–2572, Jul. 2018.

[20] N. Sünderhauf, et al., “Place Recognition with ConvNet Features: Static vs Dynamic Environments,” *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2225–2232, Jul. 2018.

[21] D. Kim, J. Park, and S. Lee, “Semantic-Aware Change Detection for Long-Term SLAM,” *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, pp. 320–327, 2019.